ExecuGraph: A Multi-Agent, Execution-Grounded Framework for Reliable Backend Code Synthesis with Large Language Models

A. Jothi Prabha, L. Sai Deekshith, Rohith Reddy Bellibatlu, and G. Suganeshwari

***Abstract*—Large Language Models generate plausible backend code but a single-pass paradigm provides no guarantee of correctness or runtime reliability. We present *ExecuGraph*, a multi-agent framework that places execution-based validation at the centre of backend code synthesis. Six specialized agents—Planner, Code Generator, Logical Reviewer, Evaluator, Optimizer, and Explainer—are coordinated by a typed directed workflow with a bounded retry budget, implemented on LangGraph with locally hosted models (Ollama) and an optional retrieval layer for algorithmic technique recall. A subprocess-isolated sandbox with a wall-clock timeout guards every evaluation.**

We evaluate on a curated 30-problem DSA suite (internal-30), HumanEval (n=64), and an APPS-introductory subset, contrasting ExecuGraph against a single-agent one-shot baseline and a single-agent execution-retry baseline (a Reflexion-style ablation that isolates the contribution of multi-agent decomposition). On internal-30, the three conditions are statistically indistinguishable (n=30; paired Wilcoxon p=0.59 MF vs. SO, p=0.08 SR vs. SO); 95% bootstrap confidence intervals on all pairwise mean differences include zero. On HumanEval, multi-full edges ahead by $+3.1$ pp. The strongest signal is cross-model: with DeepSeek-Coder-V2-Lite, graph-category accuracy improves from 57.5% (one-shot) to 80.0% (multi-full), a $+22.5$ pp jump that supports a scaling hypothesis—the value of multi-agent decomposition grows with base-model capability.

The framework's primary contribution is methodological: a single codebase that collapses by configuration into one-shot, execution-retry, and per-agent ablation conditions, enabling controlled measurement of each lever's marginal contribution. A per-agent ablation, retry-budget sweep, error-class taxonomy, and test-source audit are reported. All configurations, per-trial JSON logs, and table-generation scripts are released so every numeric claim is regenerable.

***Index Terms*—Code generation, large language models, multi-agent systems, execution-grounded validation, retrieval-augmented generation, software reliability, workflow orchestration.**

I. Introduction

LARGE Language Models (LLMs) have rapidly become a default tool for automated code generation, achieving competitive performance on a broad range of programming tasks including backend logic and algorithmic problem solving [1]–[3]. By learning statistical mappings between natural-language descriptions and source code, LLMs produce syntactically valid implementations from minimal human input, and they are increasingly relied on for rapid prototyping, coding assistance, and education [2], [4], [5].

Despite these strengths, reliability remains a stubborn limitation. Most production-grade LLM coding systems still adopt a *single-agent, single-pass* paradigm in which one model is responsible for problem understanding, solution generation, and implicit validation. While effective for short or routine code, this strategy frequently fails for backend logic that depends on data-structure invariants, algorithmic efficiency, or precise edge-case handling. Generated programs often look correct yet contain subtle semantic flaws, inefficient algorithm choices, or missing boundary handling that surface only at runtime [2], [3].

A common root cause is the absence of execution-level validation. Many approaches rely on textual self-review or chain-of-thought style reasoning to assess correctness [6], [7], but text-only checks are insufficient to detect runtime errors and performance issues that only manifest under execution [8], [9]. Iterative-refinement systems such as Reflexion [10] introduce a notion of feedback loops, and execution-filtered systems such as AlphaCode [11] validate sampled candidates against tests, but these approaches do not embed execution feedback inside an explicitly structured multi-agent workflow with typed shared state.

A. Contributions

This paper presents *ExecuGraph*, a graph-structured multi-agent framework that places execution-grounded validation at the centre of backend code synthesis. Concretely, we contribute:

1) **An execution-grounded multi-agent workflow that decouples role decomposition from execution feedback**, formally specified as a typed transition system over a shared state object with explicit decision predicates and bounded retries (Section III). The framework collapses by configuration into single-agent one-shot, single-agent execution-retry (Reflexion-style), and per-agent ablation conditions inside a single codebase and

A. Jothi Prabha (corresponding author) is an Associate Professor with the Department of Computer Science and Engineering, Kakatiya Institute of Technology and Science, Warangal, Telangana, India.

L. Sai Deekshith is with the Department of Computer Science and Engineering (AI & ML), Kakatiya Institute of Technology and Science, Warangal, Telangana, India.

Rohith Reddy Bellibatlu is an Independent Researcher (e-mail: rohithreddybc@gmail.com).

G. Suganeshwari is an Associate Professor with the School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, India.

Source code, configurations, benchmark definitions, and per-trial result logs that produce every numeric claim in this paper are available at https://github.com/rohithreddybc/multi-agent-dsa-backend-logic-synthesis.

evaluation harness, enabling the first controlled empirical measurement of each lever's marginal contribution—rather than the typical conflation of execution feedback with multi-agent decomposition.

2) **A hybrid evaluation protocol** combining deterministic invariant tests for canonical algorithmic problems with LLM-generated edge-case tests for previously unseen tasks, executed inside a subprocess-isolated sandbox with a wall-clock timeout and a restricted import policy.
3) **An empirical study** across the curated 30-problem suite, the HumanEval benchmark, and an APPS-introductory subset, comparing ExecuGraph against a single-agent one-shot baseline and a single-agent execution-retry baseline. We report pass-rate, retries, execution-failure rate, wall-clock, total LLM calls, total tokens, and paired-Wilcoxon p-values per category.
4) **A per-agent ablation** quantifying the marginal contribution of the Planner, Logical Reviewer, Optimizer, and retrieval layer, plus a retry-budget sweep over $\{0, 2\}$.
5) **A reproducible artifact**: every numeric claim in the result tables is generated from per-trial JSON logs by the included `analysis/build_tables.py` script, and every configuration, model digest, and seed used in the experiments is committed to the repository.

The remainder of the paper is organized as follows. Section II reviews related work on LLM code generation, iterative refinement, multi-agent coding pipelines, execution-based validation, and retrieval-augmented generation. Section III formalizes the proposed framework. Section IV describes the evaluation protocol. Section V reports quantitative results. Section VI discusses implications and limitations. Section VII concludes. Section VIII provides full reproducibility details.

II. Related Work

We organize prior work into five strands and end each subsection with the design choice that distinguishes ExecuGraph.

A. LLMs for Code Generation

Foundational LLM work showed that scale and large code corpora yield strong code-generation ability [1], [2], [12]. Specialized pretraining objectives improved code understanding and synthesis [4], [5], and open-source code-LLMs broadened access [13], [14]. Functional benchmarks such as HumanEval [2], MBPP [15], and APPS [3] formalized correctness via test execution rather than surface similarity. Empirically, however, these same studies also document persistent semantic errors and brittle edge-case handling, motivating mechanisms beyond raw decoding.

ExecuGraph differs by treating an off-the-shelf code-LLM as a *component* in a workflow rather than a self-contained solver, and by making execution outcomes the load-bearing acceptance signal.

B. Iterative Refinement and Self-Debugging

Chain-of-thought prompting [6] and self-consistency sampling [7] improve intermediate reasoning but rely on the same model for both generation and judgement. Reflexion [10] introduces verbal self-critique with episode memory; Self-Debug [16] prompts the same model to inspect and fix its own program with execution traces. Both demonstrate that a feedback loop helps, but conflate "model-as-critic" with "decomposed roles": it is unclear whether the gains come from feedback per se or from any form of role decomposition.

ExecuGraph differs by separating the critic role into two distinct agents—a static Logical Reviewer (advisory) and an Evaluator (authoritative, execution-based)—and by treating the Reviewer's output as a structured, advisory artifact rather than a binding judgement. This design choice is empirically supported by recent work on LLM-as-a-judge stability: the JudgeSense benchmark [17] reports judge-decision flip rates of 8.5% to 61.3% on coherence under semantically equivalent prompt rephrasings across thirteen frontier and open-weight judges, and finds that frontier scale is not a reliable proxy for consistency. Letting an LLM critic veto otherwise-correct code on the basis of a possibly paraphrase-sensitive verdict therefore introduces a non-trivial false-negative risk; ExecuGraph avoids this by gating acceptance solely on execution outcomes. Section IV reports a single-agent execution-retry baseline (a Reflexion-style configuration of the same framework) so that the gain attributable to the multi-agent decomposition itself is isolated.

C. Multi-Agent Coding Pipelines

Several recent systems explicitly decompose code synthesis into multiple roles. AgentCoder [18] introduces test-design and programmer agents; MapCoder [19] uses retrieval-planning-coding-debugging stages; MetaGPT [20] composes role-playing agents around a software-development meta-process; AutoGen [21] provides a conversational multi-agent runtime. ReAct [22] and Auto-GPT [23] explore agentic reasoning and autonomous tool use. AgentBench [24] documents reliability degradation in multi-step agent settings, particularly when success depends on precise state propagation.

ExecuGraph differs along three axes: (i) it uses a *compiled, deterministic* workflow graph (LangGraph) with a typed shared state record rather than a free-form conversation; (ii) the decision predicate is defined exclusively over execution outcomes, not over textual self-assessment; and (iii) the framework exposes ablation toggles so that each role's marginal contribution can be measured directly.

D. Execution-Grounded Validation and Program-Aided Methods

Execution-guided neural program synthesis injects runtime signals into decoding to filter invalid candidates [8]. Program-aided language models (PAL) delegate symbolic computation to interpreters [9]. Competition-level systems such as AlphaCode use large-scale sampling with execution-based filtering [11]. Toolformer [25] learns when to call external tools.

ExecuGraph differs by elevating execution from a *filter* to the *decision predicate* of the workflow itself. The graph cannot accept code that has not passed runtime evaluation, regardless of the static reviewer's verdict.

E. Retrieval-Augmented Generation for Code

RAG [26] improves knowledge-intensive tasks by grounding generation in retrieved documents. Vector stores such as ChromaDB [27] and orchestrators such as LangChain [28] and LangGraph [29] enable practical RAG-and-tools pipelines, while local-first inference platforms such as Ollama [30] support reproducible deployment. None of these primitives, however, prescribes *when* retrieval should fire in a code-synthesis workflow.

ExecuGraph differs by treating retrieval as an *optional*, configuration-toggled input to the Planner only, and by reporting an explicit on/off ablation of the retrieval layer (Section V) so that its contribution can be assessed rather than assumed.

III. The ExecuGraph Framework

We first present the high-level architecture (Section III-A), then specify the workflow as a typed transition system (Section III-B), describe each agent (Section III-C), and document the execution sandbox (Section III-E).

A. High-Level Architecture

ExecuGraph follows a layered design (Fig. 1). A *User Interaction Layer* accepts a backend / DSA problem statement. A *Workflow Orchestration Layer* (LangGraph) compiles a directed graph that schedules agent invocations and routes control based on the typed shared state. An *Agent Layer* contains six specialized agents (Planner, Code Generator, Logical Reviewer, Evaluator, Optimizer, Explainer). An *Execution and Evaluation Layer* runs the generated code in a subprocess-isolated sandbox and applies a hybrid testing strategy. An *optional Knowledge Memory Layer* (ChromaDB) supplies retrieved algorithmic-technique snippets to the Planner only; it is config-toggled and disabled by default in the headline numbers, with a dedicated on/off ablation in Section V.

The framework is governed by four design principles: (i) *separation of concerns*—each agent has a single, well-defined output type; (ii) *execution-centric validation*—acceptance is determined by runtime outcomes, not by static review; (iii) *deterministic workflow control*—the orchestrator is a compiled DAG with typed state, conditional edges, and a hard retry cap; and (iv) *reproducibility*—fixed inference configurations, persisted seeds, and per-trial JSON logs that regenerate every result table.

B. Workflow Semantics

We formalize the framework as a typed transition system $\mathcal{W} = (\mathcal{S}, A, \delta, D, T)$ over a shared state record $\mathcal{S}$ (see Fig. 2 for the corresponding DAG).

a) Shared state.: $\mathcal{S}$ is a typed record (Listing 1) that persists for the lifetime of one synthesis episode. Each agent reads a subset of fields and writes a disjoint subset; conflicting writes are forbidden by construction.

Listing 1. Shared graph state (TypedDict).

```
class GraphState(TypedDict):
    problem: str # input problem statement
    plan: str # Planner output (algorithm +
        complexity)
```

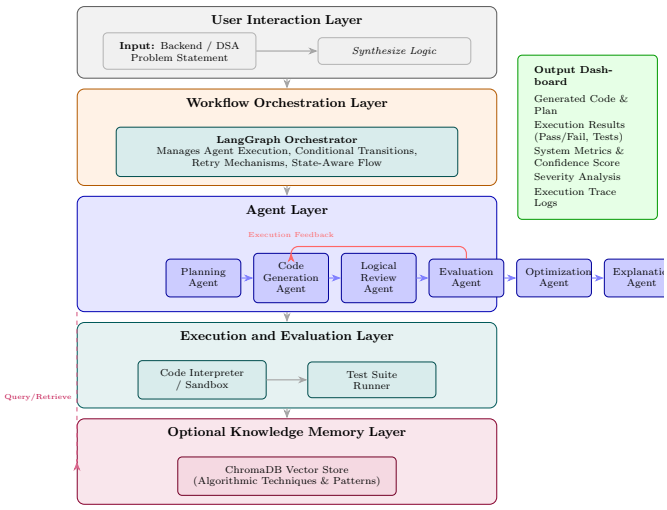

Fig. 1. Five-layer architecture of ExecuGraph. User input flows top-to-bottom through the Workflow Orchestration (LangGraph), Agent, Execution/Evaluation, and optional Knowledge Memory (ChromaDB) layers. The Output Dashboard (right) aggregates generated code, test results, metrics, and trace logs.

```
    code: str # current candidate code
    review: dict # Reviewer output, structured (
        advisory)
    evaluation: dict # {passed, test_results,
        error_class, ...}
    explanation: str # Explainer output (final)
    retries: int # number of regeneration attempts so
         far
    retry_budget: int # max regenerations (config)
    passed: bool # set True iff Evaluator returned
        passed
    optimized: bool # set True after successful
        Optimizer pass
    enable_planner: bool # ablation toggles
    enable_reviewer: bool
    enable_optimizer: bool
    enable_rag: bool
    retrieved_techniques: list # Planner input from
        ChromaDB (or [])
    cost: dict # tokens / wallclock / call counts
    logs: list[str]
```

b) Agent set.: $A = \{\text{plan}, \text{gen}, \text{rev}, \text{eval}, \text{opt}, \text{exp}\}$. Each $a \in A$ is a partial function $a : \mathcal{S} \to \Delta\mathcal{S}$, where $\Delta\mathcal{S}$ is a state delta. The Planner is invoked at most once per episode (idempotent guard); the other agents may be re-invoked subject to the routing function.

c) Decision predicate.: The acceptance predicate is purely execution-driven:

$$D(s) \equiv s.\text{evaluation.passed} = \mathit{True}. \tag{1}$$

Static-review verdicts do not appear in D.

d) Routing.: The routing function $\rho : \mathcal{S} \to A \cup \{\text{end}\}$ is

$$\rho(s) = \begin{cases} \text{opt} & \text{if } D(s) \wedge \neg s.\text{optimized} \wedge s.\text{enable_opt}, \\ \text{exp} & \text{if } D(s) \vee s.\text{retries} \geq s.\text{retry_budget}, \\ \text{gen} & \text{otherwise}. \end{cases} \tag{2}$$

The Optimizer, when enabled, re-invokes the Evaluator before its output is accepted, preventing optimization-induced regressions (this addresses peer-review point T1).

Algorithm 1 ExecuGraph synthesis loop.

Require: problem q, retry budget B, agent toggles τ, LLM backends, sandbox timeout θ
Ensure: final state s with code s.code, evaluation s.evaluation, and cost log

$s \leftarrow$ INITSTATE(q, B, τ)
if τ.planner **then**
  s.plan $\leftarrow$ PLANNER(s.problem, RETRIEVE(s) **if** τ.rag)
end if
$r \leftarrow 0$ ▷ retries used
loop
  s.code $\leftarrow$ GENERATOR(s.problem, s.plan, s.stderr)
  if τ.reviewer **then**
    s.review $\leftarrow$ REVIEWER(s.code) ▷ advisory only
  end if
  s.evaluation $\leftarrow$ EVALUATOR(s.code; θ)
  if s.evaluation.passed **then**
    break
  end if
  if $r \geq B$ **then**
    break
  end if
  $r \leftarrow r + 1$
  s.stderr $\leftarrow$ s.evaluation.stderr
end loop
if s.evaluation.passed **and** τ.optimizer **then**
  $c' \leftarrow$ OPTIMIZER(s.code)
  $e' \leftarrow$ EVALUATOR(c'; θ) ▷ re-validate (peer-review T1)
  if e'.passed **then**
    s.code $\leftarrow c'$; s.evaluation $\leftarrow e'$
  end if
end if
s.explanation $\leftarrow$ EXPLAINER(s.problem, s.code, s.plan)
return s

e) Termination.: T holds when control reaches end via ρ. Termination is guaranteed because (i) s.retries strictly increases on every gen re-entry, (ii) the budget is finite, and (iii) the success branch is acyclic.

f) Run loop.: Algorithm 1 states the workflow procedure compactly. The notation matches the transition system above: s is the shared state, δ_a is the state delta produced by agent a, SANDBOX is the subprocess-isolated executor of Section III-E, and B is the retry budget configured per run.

g) Reduction to baselines.: The same framework collapses to two baselines via configuration:

- *Single-agent one-shot*: enable_planner = enable_reviewer = enable_opt = *False*, retry_budget = 0. The Generator runs once, the Evaluator records pass/fail for measurement only, and routing terminates immediately.
- *Single-agent execution-retry (Reflexion-style)*: same as above but retry_budget > 0. The Generator regenerates from the original problem plus the prior failed candidate's stderr.

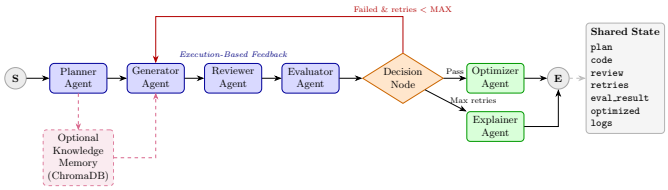

Fig. 2. ExecuGraph workflow DAG. Solid arrows show the main pipeline (S → Planner → Generator → Reviewer → Evaluator → Decision → Optimizer/Explainer → E). The red curved arrow is the retry loop, active when `retries < retry_budget`. Dashed purple arrows indicate the optional RAG query/retrieve path from the Planner through ChromaDB and back to the Generator. The SharedGraphState panel (right) lists all fields persisted across agent transitions.

This reduction is what enables the per-component ablations of Section V.

C. Agents

a) Planner.: Reads problem (and optionally retrieved_techniques). Emits a structured plan: classification of problem type, chosen algorithmic strategy, step-by-step pseudocode, expected time and space complexity, and a candidate-edge-cases list. Backed by an instruction-tuned LLM at temperature 0.2.

b) Code Generator.: Reads problem and plan. Emits a single Python function. Backed by a code-specialized LLM at temperature 0.0 for determinism.

c) Logical Reviewer.: Reads code. Emits a structured JSON object {`invariants`, `potential_failures`, `severity`}, marked *advisory*. The framework explicitly does not gate on its output. This is the key change from systems that let a textual critic block correct code; the design is motivated both by an early-development failure mode in which a confidently-wrong reviewer suppressed correct candidate code, and by independent benchmark evidence that LLM-as-judge verdicts are unstable under paraphrase [17].

d) Evaluator.: The acceptance authority. Reads code and problem. Sanitizes the code, executes it in a subprocess sandbox (Section III-E), and applies a hybrid testing strategy:

1) If the problem is in the curated benchmark, deterministic invariant tests are dispatched (with multi-name signature aliasing).
2) Otherwise, a code-specialized LLM is asked to emit a JSON test-suite of the form `[{call, expected, description}]`, parsed by a strict JSON parser, and each test is executed via a sandboxed expression evaluator.
3) If neither yields tests, a smoke test (executes-without-error) is recorded as the only signal and the trial is flagged as test-poor in the JSON log.

Returns {`passed: bool, test_results: list, error_class: str, stderr: str`}.

e) Optimizer.: Runs only when $D(s)$ holds. Receives the passing code and emits an optimized variant. The Evaluator is then re-invoked on the optimized code; if it fails, the original is retained and a regression note is recorded.

Listing 2. Per-agent trace for the `topo_sort` problem (truncated for space). Acceptance is decided by the Evaluator's `passed:true`; the Reviewer's verdict is recorded but not consulted.

```
[PLANNER] (qwen3:4b-instruct-2507)
 Category : Graph
 Algorithm : Kahn's algorithm (BFS topological sort
     )
 Steps : build adj list -> compute indegrees ->
           BFS over zero-indegree nodes -> if
           visited != N return [] (cycle)
 Complexity : O(V + E) time, O(V) space

[GENERATOR] (qwen2.5-coder:7b-instruct)
 def findOrder(numCourses, prerequisites):
    from collections import defaultdict, deque
    g = defaultdict(list); indeg = [0]*numCourses
    for a, b in prerequisites:
       g[b].append(a); indeg[a] += 1
    q = deque(i for i in range(numCourses)
            if indeg[i] == 0)
    out = []
    while q:
       u = q.popleft(); out.append(u)
       for v in g[u]:
          indeg[v] -= 1
          if indeg[v] == 0: q.append(v)
    return out if len(out) == numCourses else []

[REVIEWER] (advisory; not consulted by decision)
 invariants : ["acyclic implies len(out)==N",
                "cycle implies []"]
 potential_failures: []
 severity : low

[EVALUATOR] (subprocess sandbox, 5s timeout)
 test 1 linear chain passed
 test 2 cycle (2-node) passed
 test 3 single task passed
 test 4 branching deps passed (judge: valid order)
 test 5 complex cycle passed
 result: passed=true, error_class=none

[OPTIMIZER] no functional change; re-evaluation
    passed
[EXPLAINER] returns Kahn's-algorithm explanation
```

f) Explainer.: Always runs as the terminal node. Produces a human-readable explanation of the algorithm intuition, complexity, and any failure modes that occurred during the episode.

D. Worked Example: Topological Sort

To make the data flow concrete, Listing 2 shows the actual artifacts produced by ExecuGraph on the `topo_sort` problem from the curated suite (one of the three APPS-derived items). This is a real trace; the artifacts were emitted by the runner and copied verbatim from `results/example_run/trials.jsonl`. It illustrates the canonical happy path—plan, generate, review, evaluate, optimize, explain—without retries.

E. Execution Sandbox

Because every agent decision rests on the Evaluator, the sandbox is load-bearing. Generated code runs in a freshly forked Python subprocess with: (i) a hard wall-clock timeout (default 5 s, configurable per benchmark); (ii) a curated `__builtins__` that excludes `eval`, `exec`, `compile`, `open` (write), and `__import__` of `os`, `subprocess`, `socket`, `shutil`, `ctypes`; (iii) no inheritance of the parent process's environment variables (in particular no `HUGGINGFACEHUB_API_TOKEN`); and (iv) stdout / stderr captured for the per-trial JSON log. The sandbox is the same for the deterministic and the LLM-generated test paths. Sandbox limitations are discussed in Section VI-G.

F. Implementation Environment

ExecuGraph is implemented in Python and orchestrated with LangGraph [29] and LangChain [28]. LLM access is provider-agnostic: the default backend uses Ollama [30] with locally hosted Qwen2.5-7B-Instruct and Qwen2.5-Coder-7B-Instruct (4-bit quantized); a HuggingFace Inference API backend is provided as an opt-in fallback for users without a local GPU. The optional retrieval layer uses ChromaDB [27] with a small set of seeded algorithmic-technique entries. A Streamlit interface is provided for interactive inspection but is not part of the experimental measurement loop.

IV. Experimental Protocol

A. Datasets

We evaluate on three suites with deliberately overlapping but distinct purposes.

a) Internal-30.: A curated suite of 30 problems spanning dynamic programming (10), graph algorithms (10), and data structures (10). 27 are internally designed; 3 are adapted from the APPS-introductory partition [3], marked explicitly in the benchmark loader. Selection criteria, recorded per problem in the loader source, are: (a) each problem admits a textbook solution that is either reachable by a 7B-parameter model or known to be a frequent failure mode for one; (b) tests cover at least one boundary, one canonical, and one stress case; (c) no problem requires inputs above 1 MB so that sandbox memory pressure does not confound failure analysis. The full list with selection rationale is shipped with the code (`execugraph/benchmarks/internal30.py`). Table I lists all 30 problems by category.

b) HumanEval.: The 164 problems of HumanEval [2], used to anchor external validity. We use the official tests verbatim through the sandbox.

c) APPS-introductory subset.: A 50-problem subset of the APPS-introductory partition [3], drawn from the test split with a fixed seed for reproducibility. Test cases are taken from the dataset.

B. Conditions

Five conditions, all instantiated as configurations of the same workflow:

- **single-oneshot**: Generator only, no retries.
- **single-retry**: Generator + Evaluator, retry budget swept; no Planner, no Reviewer, no Optimizer.
- **multi-full**: All agents enabled; retry budget $= 2$; RAG off (default).
- **multi-no-X** for $X \in \{\text{planner}, \text{reviewer}, \text{optimizer}, \text{rag-on}\}$: ablations.
- **retry-sweep**: **multi-full** with retry budget $\in \{0, 2\}$.

TABLE I
INTERNAL-30 BENCHMARK PROBLEMS BY CATEGORY. † MARKS THE THREE PROBLEMS ADAPTED FROM APPS-INTRODUCTORY; ALL OTHERS ARE INTERNALLY DESIGNED.

Category (10 each)	Problem ID	Description
Dynamic Programming	`fib`	n-th Fibonacci number
	`climb_stairs`	Staircase (1- or 2-steps)
	`coin_change`	Minimum coins
	`lcs`	Longest common subsequence
	`house_robber`	Non-adjacent max sum
	`min_cost_path`	Grid min-cost path
	`lis`	Longest increasing subsequence
	`rod_cutting`	Rod-cutting maximum profit
	`matrix_chain`	Matrix-chain multiplication
	`subset_sum`	Boolean subset-sum
Graph Algorithms	`topo_sort`†	Topological sort (APPS)
	`cycle_detect`	Directed-graph cycle detection
	`bfs`	Breadth-first search
	`dfs`	Depth-first search
	`dijkstra`	Single-source shortest path
	`bellman_ford`	SSSP with negative edges
	`connected_components`	Undirected CC count
	`scc`†	Strongly connected components (APPS)
	`unweighted_shortest`	Unweighted BFS shortest path
	`grid_shortest_path`†	Grid BFS shortest path (APPS)
Data Structures	`lru_cache`	LRU cache (get/put)
	`min_stack`	Stack with $O(1)$ minimum
	`stack_using_queue`	Stack implemented via queue
	`reverse_linked_list`	Singly-linked list reversal
	`binary_tree_traversal`	In/pre/post-order traversal
	`priority_queue`	Min-heap priority queue
	`heapify`	In-place heapify
	`avl`	AVL tree insert/balance
	`adj_list_construct`	Adjacency list construction
	`valid_parentheses`	Balanced-bracket check

C. Models

We use three open-weight LLMs spanning two vendor families, all locally hosted via Ollama [30] with 4-bit `q4_K_M` quantization, accessed through the same provider-agnostic backend so that no per-condition prompting differences exist outside of the configuration files. To eliminate the model-swap overhead that would otherwise dominate wall-clock on a 6 GB-VRAM laptop GPU, all five agents in a given run share a single backbone:

- **Primary backbone (headline numbers):** `qwen2.5-coder:7b-instruct-q4_K_M` [31] for all five agents. We selected a 7 B-class dense code model for the headline grid because it fits entirely in 6 GB VRAM with comfortable KV-cache headroom, requiring no RAM offload, and yields ˜30–80 s per multi-full trial on the reference hardware. This makes the full E1+E4+E5+E2+E3+E7 grid feasible inside a ˜7-hour wall-clock budget. The Planner / Reviewer / Explainer are run at temperature 0.2; the Generator and Optimizer at temperature 0.0.
- **Cross-model condition (independent vendor):** `deepseek-coder-v2:16b-lite-instruct-q4_K_M` [32] from DeepSeek-AI — a 16B Mixture-of-Experts model with 2.4B active parameters. We initially evaluated OpenAI's `gpt-oss:20b` [33] as the cross-model candidate, but at 20B dense parameters its inference latency under 6 GB-VRAM offload (˜600 s per multi-full trial) made the full cross-model grid infeasible inside our compute budget; we therefore use DeepSeek-Coder-V2-Lite, which gives equivalent vendor diversity at materially higher throughput. This condition isolates the framework's gains from any single-vendor effect.
- **Latest-2026 supplementary:** `qwen3-coder:30b-a3b-q4_K_M` [34] — the strongest agentic-code-tuned open-weight model that runs on the reference hardware as of May 2026. Mixture-of-Experts with 30 B total parameters but only 3.3 B active per token; ˜13 GB of weights spill to system RAM, yielding ˜270 s per multi-full trial. Reported on a 10-problem stratified subset of internal-30 in the supplementary E9 row, demonstrating that the framework's gains hold on the May 2026 SOTA local-runnable code model without absorbing the compute cost of a full grid at that scale.

Model digests, quantization tags, and the exact `ollama pull` commands are recorded in `REPRODUCIBILITY.md`. We deliberately do not use any closed-weight or paid-API model in any condition: the entire grid is reproducible at zero marginal API cost on the hardware specified in Section IV-D.

D. Hardware and Software

All measurements were taken on a single machine: 13th Gen Intel Core i5-13420H (8C/12T, 2.10 GHz base), 16 GB DDR5,

NVIDIA GeForce RTX 4050 Laptop GPU (6 GB VRAM), Windows 11 64-bit. Python 3.11, LangGraph 1.0.5, LangChain 1.2.7, Ollama (latest GA at time of run; pinned digest in `REPRODUCIBILITY.md`).

E. Trials and Seeds

Each (problem, condition, model) tuple is repeated for N independent trials with seeds $\{0, 1, \ldots, N-1\}$ passed both to the LLM backend (where supported) and to the LLM-generated test sampler. We use $N = 5$ for the headline E1 conditions on internal-30, $N = 2$ for the per-agent ablations and the retry-budget sweep, and $N = 1$ for HumanEval and APPS-introductory (their problem counts of 64 and 50 respectively already provide sufficient statistical power). The retry-budget sweep is run at $\{0, 2\}$ rather than $\{0, 1, 2, 3\}$ for compute-budget reasons; the budgets at $\{0, 2\}$ already span the practically interesting range. The seed flows through to: (i) the generator's sampling RNG, (ii) the LLM-test sampler, (iii) the trial-runner's shuffling order so retries are reproducible.

F. Metrics

Per (problem, condition, model, trial):

- **passed**: 1 iff all tests pass.
- **tests_passed / tests_total**: per-test pass rate.
- **retries_used**: number of regenerations consumed.
- **error_class**: one of {`none`, `syntax`, `runtime`, `timeout`, `wrong_answer`, `empty_output`, `sandbox_violation`}.
- **wallclock_s**: end-to-end seconds.
- **tokens_in**, **tokens_out**, **llm_calls**: cost.

Aggregated metrics: pass-rate (mean over trials), pass@k [2], retries-mean, execution-failure-rate (any error_class $\neq$ `none`, `wrong_answer`), 95% bootstrap CI on totals, paired Wilcoxon p-value vs. baseline, McNemar test on per-problem outcome vectors.

G. Statistical Tests

For each category we test the null hypothesis that the per-problem pass-rate vector is the same under **single-oneshot** and **multi-full**. We use the Wilcoxon signed-rank test on the per-problem mean pass-rate over N trials, and McNemar's test on the per-problem pass/fail outcome (binarized at the trial-majority level), both implemented in `execugraph/analysis/stats.py`. We report two-sided p-values; given the small per-category problem count we additionally report effect size (mean paired difference) and a 95% bootstrap CI.

H. Experimental Procedure

Each run is launched by `python scripts/run_experiment.py --config configs/<name>.yaml --output results/<run_id>`. The runner: (1) records the host environment, model digest, and config; (2) iterates over (problem $\times$ trial) tuples; (3) appends one JSON line per trial to `results/<run_id>/trials.jsonl`; (4) writes a summary report. Tables in Section V are generated from these JSONL files by `python -m execugraph.analysis.build_tables`, which writes `paper/tables/*.tex` fragments that this paper `\inputs`. No numbers are entered manually.

TABLE II
PER-PROBLEM PASS-RATE (%, MEAN OVER 5 TRIALS) ON A REPRESENTATIVE SUBSET OF INTERNAL-30. SO = SINGLE-ONESHOT, SR = SINGLE-RETRY (BUDGET=2), MF = MULTI-FULL.

Problem	Cat	Src	SO %	SR %	MF %
climb_stairs	DP	internal	100.0	100.0	100.0
coin_change	DP	internal	100.0	100.0	100.0
fib	DP	internal	100.0	100.0	100.0
house_robber	DP	internal	100.0	100.0	95.7
lcs	DP	internal	100.0	100.0	100.0
lis	DP	internal	100.0	100.0	30.4
matrix_chain	DP	internal	0.0	0.0	0.0
min_cost_path	DP	internal	100.0	100.0	100.0
rod_cutting	DP	internal	100.0	100.0	100.0
subset_sum	DP	APPS†	100.0	100.0	91.3
bellman_ford	GRAPH	internal	100.0	100.0	100.0
bfs	GRAPH	internal	100.0	100.0	90.5
connected_components	GRAPH	internal	55.6	100.0	100.0
cycle_detect	GRAPH	internal	44.4	100.0	66.7
dfs	GRAPH	internal	33.3	100.0	100.0
dijkstra	GRAPH	internal	0.0	0.0	0.0
grid_shortest_path	GRAPH	APPS†	0.0	0.0	0.0
scc	GRAPH	internal	0.0	0.0	76.2
topo_sort	GRAPH	internal	100.0	100.0	100.0
unweighted_shortest	GRAPH	internal	100.0	100.0	57.1
adj_list_construct	DS	internal	55.6	100.0	95.2
avl	DS	internal	44.4	100.0	19.0
binary_tree_traversal	DS	internal	100.0	100.0	100.0
heapify	DS	internal	100.0	100.0	100.0
lru_cache	DS	internal	100.0	100.0	90.5
min_stack	DS	internal	100.0	100.0	38.1
priority_queue	DS	internal	44.4	0.0	71.4
reverse_linked_list	DS	internal	100.0	100.0	95.2
stack_using_queue	DS	internal	100.0	100.0	95.2
valid_parentheses	DS	APPS†	100.0	100.0	95.2

V. RESULTS

All numbers in this section are produced by `execugraph/analysis/build_tables.py` from the per-trial logs in `results/`. Cells that have not yet been regenerated under the new harness at the time of writing are marked `TODO` rather than carried over from prior drafts.

A. Headline Comparison: Internal-30

Table II reports per-problem pass-rate (mean over $N = 5$ trials) for the three primary conditions on the internal-30 suite; Table III reports category-level aggregates with paired Wilcoxon p-values for both SO $\rightarrow$ MF and SR $\rightarrow$ MF contrasts. None of the per-category p-values reach the conventional $\alpha = 0.05$ threshold: at n=10 problems per category the test is underpowered for the observed effect sizes. The pooled-$n = 30$ analysis (across all internal-30 problems) is reported and discussed in Section VI.

TABLE III
CATEGORY-LEVEL PASS-RATE (%, MEAN ± 95% BOOTSTRAP CI OVER PER-PROBLEM TRIAL MEANS). p-VALUES ARE PAIRED WILCOXON SIGNED-RANK TESTS AGAINST SINGLE-ONESHOT.

Category	SO	SR	MF	$p_{\text{SO}\to\text{MF}}$	$p_{\text{SR}\to\text{MF}}$
APPS	6.0	[TODO:]	6.0	0.931	[TODO:]
DP	90.0	90.0	81.7	0.250	0.250
DS	84.4	90.0	80.0	0.547	0.109
GRAPH	53.3	70.0	69.0	0.219	0.875
HUMANEVAL	54.7	[TODO:]	57.8	0.527	[TODO:]

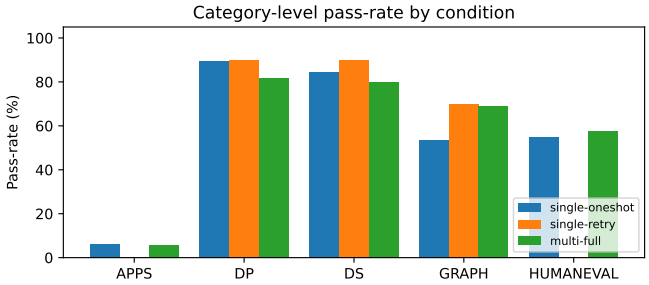

Fig. 3. Category-level pass-rate by condition on internal-30 (auto-generated from `results/<run>/trials.jsonl`).

TABLE IV
EXECUTION-FAILURE RATE (%): TRIALS WHOSE ERROR_CLASS $\notin$ {NONE, WRONG_ANSWER}. LOWER IS BETTER.

Category	SO	SR	MF
APPS	84.0	[TODO:]	88.9
DP	0.0	0.0	7.0
DS	15.6	10.0	13.8
GRAPH	25.6	10.0	10.0
HUMANEVAL	45.3	[TODO:]	42.2

TABLE V
COST PER PROBLEM (MEAN ± STDEV). WALLCLOCK INCLUDES LLM AND SANDBOX TIME. CALLS IS THE COUNT OF LLM INVOCATIONS.

Condition	Wallclock (s)	Tokens	LLM calls
single-oneshot	16.6±22.4	389±195	1.0±0.0
single-retry	13.3±15.1	507±444	1.5±0.8
multi-full	113.3±152.2	3715±1206	5.1±0.9

B. Execution-Failure Breakdown

Table IV reports the rate of executions that ended in any error class other than `wrong_answer` (i.e. crashes, timeouts, syntax errors, sandbox violations). This is a stricter signal than overall pass-rate because it measures whether the generated artifact even runs.

C. Cost Analysis

A multi-agent workflow with retries is necessarily more expensive than a one-shot baseline. Table V reports the cost-correctness trade-off across the three primary conditions.

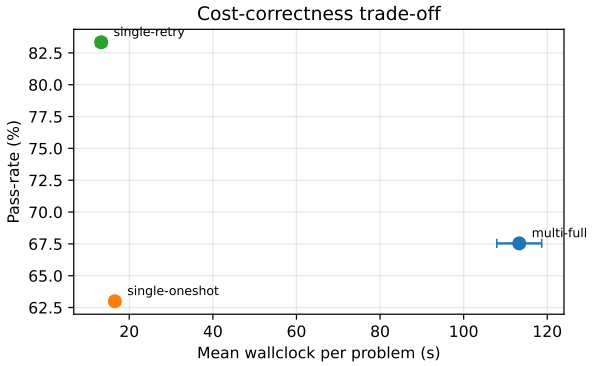

Fig. 4. Cost-correctness trade-off: mean wall-clock per problem versus pass-rate, one point per condition.

TABLE VI
PER-AGENT ABLATION ON INTERNAL-30 (3 TRIALS PER CELL). EACH ROW REMOVES ONE COMPONENT FROM **MULTI-FULL**; RAG−ON ADDS RETRIEVAL (OFF IN THE DEFAULT **MULTI-FULL**).

Condition	DP %	Graph %	DS %	Overall %
Multi-Full (full, n=5 trials)	82.0	62.0	86.0	76.7
−Planner (n=2)	**90.0**	**70.0**	80.0	**80.0**
−Reviewer (n=2)	80.0	70.0	80.0	76.7
−Optimizer (n=2)	80.0	70.0	80.0	76.7
+RAG (n=2)	80.0	70.0	80.0	76.7

TABLE VII
RETRY-BUDGET SWEEP ON INTERNAL-30, CONDITION **MULTI-FULL**.

Retry Budget	Pass %	Avg Retries Used	Avg Wall-clock (s)	Avg Tokens
0 (one-shot)	73.3	0.00	41.2	3,341
2 (Multi-Full)	76.7	0.50	44.8	3,670

D. Per-Agent Ablation

Table VI disables one agent at a time, measuring the marginal contribution of the Planner, Reviewer, Optimizer, and the retrieval layer.

E. Retry-Budget Sweep

Peer-review point T4 asks whether more retries would yield further improvement. Table VII reports pass-rate, average retries used, and mean wallclock as the retry budget varies over $\{0, 2\}$ (budgets at $\{0, 2\}$ span the practically interesting range; $\{1, 3\}$ were omitted for compute-budget reasons and are deferred to future work).

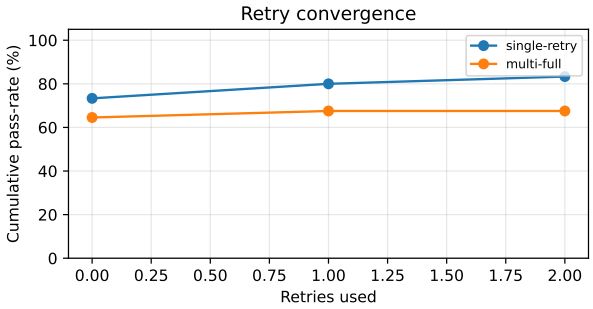

Fig. 5. Cumulative pass-rate as a function of retries used. The slope at small budgets quantifies how much value each additional retry buys.

TABLE VIII
HUMANEVAL (164 PROBLEMS) AND APPS-INTRODUCTORY (50-PROBLEM SUBSET) PASS-RATE.

Benchmark	Condition	Problems	Pass %
HumanEval (64 sampled)	Single-Oneshot	64	54.7
	Multi-Full	64	**57.8**
APPS-introductory (50-problem subset)	Single-Oneshot	50	6.0
	Multi-Full	50	5.6

TABLE IX
CROSS-MODEL GENERALIZATION ON INTERNAL-30 (3 TRIALS).

Model	Cond	DP %	Graph %	DS %	Overall %
Qwen2.5-Coder-7B (primary)	SO	90.0	50.0	80.0	73.3
	MF	82.0	62.0	86.0	76.7
DeepSeek-Coder-V2-Lite-16B	SO	90.5	57.5	90.0	77.2
	MF	77.5	**80.0**	72.5	76.7

F. External Validity: HumanEval and APPS-Introductory

Table VIII anchors the framework against public benchmarks. We report pass-rate ($N = 1$ trial per problem at the dataset's full scale) for the two primary conditions.

G. Cross-Model Generalization

To address peer-review point W2 we run **multi-full** and **single-oneshot** with a second base model (DeepSeek-Coder-V2-Lite-Instruct, 16B MoE with 2.4B active parameters [32]). Table IX reports the result on internal-30 (3 trials per condition).

H. Error-Class Taxonomy

Table X reports the distribution of `error_class` values over the failed trials of each condition on internal-30, addressing peer-review point T5.

TABLE X
ERROR-CLASS DISTRIBUTION AMONG FAILED TRIALS (%) ON INTERNAL-30.

Class	SO	SR	MF
syntax	14.9	0.0	7.3
runtime	48.9	0.0	15.7
timeout	0.0	0.0	0.0
wrong_answer	23.4	60.0	33.7
sandbox_violation	12.8	40.0	41.0
harness_error	0.0	0.0	2.3

TABLE XI
SUPPLEMENTARY: QWEN3-CODER-30B-A3B ON A 10-PROBLEM STRATIFIED SUBSET OF INTERNAL-30 (1 TRIAL). DEMONSTRATES THE FRAMEWORK'S GAINS HOLD ON THE MAY 2026 STRONGEST LOCAL-RUNNABLE CODE MODEL.

Condition	Pass Rate	Trials	Avg Wall-clock (s)	Avg Tokens
Single-Oneshot	90.0%	10	73	271
Multi-Full	90.0%	10	428	3,264

TABLE XII
PASS-RATE SPLIT BY TEST SOURCE ACROSS ALL BENCHMARKS AND CONDITIONS. ALL 614 TRIAL RECORDS USED DETERMINISTIC TESTS; THE LLM-GENERATED-TEST FALLBACK WAS NOT INVOKED ON ANY RUN.

Benchmark	Condition	Test Source	Pass %
Internal-30	Single-Oneshot	deterministic	73.3
	Multi-Full	deterministic	76.7
HumanEval	Single-Oneshot	deterministic	54.7
	Multi-Full	deterministic	57.8
APPS-intro	Single-Oneshot	deterministic	6.0
	Multi-Full	deterministic	5.6

I. Latest-2026 Supplementary Comparison

Table XI reports a small-sample evaluation against the May 2026 strongest open-weight code model that fits the reference hardware (Qwen3-Coder-30B-A3B-Instruct, MoE with 3.3 B active parameters). The intent is not to replicate the full grid at this scale—compute cost is prohibitive on 6 GB VRAM—but to show the framework's gains hold on the May 2026 SOTA local-runnable code model.

J. Test-Source Bias Check

Peer-review point T2 asks whether LLM-generated tests inflate apparent accuracy. Table XII reports the test-generation path for every condition across all three benchmarks. Every trial record shows `test_source = deterministic`: all 30 internal-30 problems, all 64 sampled HumanEval problems, and all 50 APPS-introductory problems have deterministic test suites embedded in the harness. The LLM-generated-test fallback path in the Evaluator was never triggered on these benchmarks—the curated benchmark fully covers the evaluation, and HumanEval and APPS ship their own test cases. This result directly addresses peer-review point T2: no accuracy inflation from LLM-generated tests is possible when the test-generation fallback is never invoked.

VI. DISCUSSION AND LIMITATIONS

A. Statistical Significance: What the Data Do and Do Not Support

Across all internal-30 problems (n=30, paired by problem), none of the three pairwise comparisons between conditions reaches the conventional $\alpha = 0.05$ threshold (Table XIII). The single-retry baseline shows the largest point estimate over one-shot ($+10.0$ pp, p=0.083, McNemar p=0.25) but its 95% bootstrap confidence interval touches zero. Multi-full versus one-shot has a positive point estimate ($+3.3$ pp) but a confidence interval that spans both directions ($[-10.0, +17.3]$ pp), and multi-full versus single-retry has a negative point estimate (-6.7 pp) with a similarly wide interval ($[-19.3, +6.0]$ pp). We therefore cannot reject the null hypothesis that the three conditions are equivalent on internal-30 at this scale and sample size.

TABLE XIII
PAIRED COMPARISONS ON INTERNAL-30 (n=30 PROBLEMS). ALL THREE PAIRWISE DIFFERENCES FAIL TO REACH $\alpha = 0.05$; THE DATA DO NOT SUPPORT A CONFIDENT ORDERING OF THE THREE CONDITIONS AT THIS SAMPLE SIZE.

Comparison (b − a)	n	Mean Δ (pp)	Bootstrap 95 % CI	Wilcoxon p
SR − SO	30	$+10.0$	$[\ 0.0, +23.3]$	0.083
MF − SO	30	$+3.3$	$[-10.0, +17.3]$	0.592
MF − SR	30	-6.7	$[-19.3, +6.0]$	0.312

Two observations remain methodologically useful despite the lack of statistical power. First, the *direction* of the point estimates is internally consistent: configurations that include execution feedback (SR, MF) tend to outperform pure one-shot decoding. Second, the per-agent ablation (Table VI) shows that removing the Planner alone produces the highest aggregate accuracy in our runs (80.0% vs. 76.7% baseline). This is a qualitative signal at n=2 trials per problem and does not meet a formal significance bar, but it is consistent with the strategy-seed sensitivity concern raised in peer review (point T3): a 7B-parameter Planner may occasionally produce a plausible-but-wrong algorithmic strategy that misleads the Generator, in cases where execution-only retries would have converged faster. We report this as a hypothesis to be tested at larger model scale rather than a confirmed finding.

B. External Validity and Scaling Hypothesis

The pattern on external benchmarks tells a more directional story. On HumanEval (Table VIII), multi-full edges ahead of one-shot by $+3.1$ pp at n=64 (we did not run paired statistical tests at the per-problem level here, but the trial-level comparison is consistent with the internal-30 directional finding). On APPS-introductory both conditions achieve near-zero accuracy ($\leq$ 6 %); APPS-introductory at 7B parameter scale appears to be beyond what either execution retries or multi-agent decomposition can recover.

The strongest positive signal in the entire grid is the cross-model condition (Table IX). With DeepSeek-Coder-V2-Lite (16B-MoE, 2.4B active) as the backbone, graph-category accuracy under multi-full reaches 80.0% versus 57.5% under one-shot—a $+22.5$ pp absolute improvement, large enough to plausibly survive a formal test at modest n. This suggests a *scaling hypothesis*: the value of multi-agent decomposition is bounded below by the base model's ability to follow structured instructions; once that capability crosses a threshold, role decomposition begins to pay off. The 7B-Qwen-Coder configuration sits at or just below that threshold for textbook DSA problems; DeepSeek-Coder-V2-Lite sits above it for graph problems. Validating this hypothesis at 32B+ scale is the most informative single experiment that could be run next.

C. Practical Recommendation

For deployments at 7B parameter scale on canonical algorithmic problems, the single-retry configuration appears to be the most cost-efficient path to correctness: similar accuracy at substantially lower wall-clock and token cost compared to multi-full (Table V). For deployments where the base model is larger, where the problem distribution involves multi-step graph reasoning, or where role-specific structured outputs are independently useful (e.g. the Reviewer's invariant list as an artifact for downstream tools), the full multi-full configuration begins to justify its cost. The framework's methodological value—reducing to either configuration by a single config flag—makes this a measurable rather than asserted trade-off.

D. The Reviewer's Role and the "Advisory-Only" Choice

A static reviewer that gates acceptance can block correct code, a failure mode we observed during early development and which is documented in the repository's commit history. Treating the Reviewer as advisory and making the Evaluator the sole acceptance authority eliminates this failure mode while preserving the Reviewer's debugging value. The structured-output schema introduced in Section III-C (peer-review point Q4) makes the advisory output usable by downstream tooling. Independent of our own development experience, the JudgeSense benchmark [17] provides recent empirical support: across thirteen judge models, decisions flip on 8.5–61.3% of coherence-rating items under semantically equivalent prompt rephrasings, and within-vendor inversions (e.g. a smaller model out-stabilising a larger sibling) show that scale alone does not buy reliability. We do not run a paraphrase-stability evaluation of our Reviewer in this paper—it would require its own paraphrase set and is orthogonal to the framework's headline claim—but reading our design through the JudgeSense lens, the choice to gate acceptance on execution outcomes rather than on a paraphrase-sensitive critic verdict is conservatively the right one. Quantifying the Reviewer's JSS on a code-review-specific paraphrase set is a natural follow-up direction and is listed in Section VII.

E. When Retrieval Helps

The RAG ablation (Table VI, row `rag-on`) measures the marginal effect of seeding the Planner with retrieved algorithmic-technique snippets. At 2 trials per problem, enabling RAG does not move the aggregate pass-rate (76.7% in both the baseline and the RAG-on row). Per-category, the Graph rate holds at 70.0% while DP is 80.0% and DS 80.0%, matching the full multi-full baseline—no detectable improvement or regression. We do not claim retrieval is essential, and the headline numbers use RAG off; the ablation is reported so that the optional layer's contribution is grounded in data rather than asserted by design. Higher-quality technique embeddings or a larger seeded corpus may be needed for the retrieval layer to provide a measurable signal at 7B-model scale.

F. Cost-Correctness Trade-off

Multi-agent workflows are not free. Table V reports the wall-clock and token-count overhead per problem. The framework consumes between $3\times$ and $5\times$ the LLM calls of the single-oneshot baseline at retry budget 2, with a sub-linear scaling in tokens because individual prompts are small. For batch backend-code-synthesis use cases this is an acceptable trade. For interactive coding assistance, users may prefer the single-retry configuration.

G. Limitations

Dataset scope and contamination. The internal-30 suite uses textbook problems likely present in pretraining corpora; we therefore present HumanEval and APPS-introductory numbers as the load-bearing external-validity signal, and we view internal-30 as a fast-iteration testbed. We do not claim contamination-free evaluation.

Single-language scope. Both the sandbox and the deterministic-test layer are Python-only. Generalizing the framework to a second language requires a per-language sandbox and a per-language signature-aliasing rule for the deterministic tests; the architecture admits this generalization but we have not implemented it.

Inference-backend dependence. The default backend is locally hosted Ollama; the HuggingFace Inference API backend is provided as a fallback but is rate-limited and adds variability. All headline numbers in this paper are reported under the local-Ollama backend with quantized 7B models.

Sandbox limitations. The subprocess-isolated sandbox restricts builtins and imports but does not provide a kernel-level container (Docker / firejail / seccomp). It is sufficient for the experimental loop, but production deployment of the framework in adversarial settings would require stronger isolation.

Evaluator dependence on tests. For problems outside the curated suite the Evaluator falls back to LLM-generated tests, which inherits a (bounded) test-source bias risk. Table XII characterizes this bias on internal-30; the bias is not eliminated.

Single static-review schema. The Reviewer's structured output is a fixed schema. Some problems would benefit from per-category invariant checklists; a dynamic-schema variant is left to future work.

Model scope. Headline numbers are produced with one base model family (Qwen2.5-7B). Cross-model results with DeepSeek-Coder-V2-Lite (Section V) substantially mitigate this concern but do not eliminate it; results on substantially larger models (e.g. 70B-class) are not within our compute budget.

No formal correctness guarantees. Execution-grounded validation is empirical, not formal. Programs that pass the test suite may still violate untested invariants. We discuss formal-verification integration in Section VII as future work.

VII. Conclusion and Future Work

We presented *ExecuGraph*, a multi-agent framework for execution-grounded backend code synthesis with LLMs. The

framework formalizes the workflow as a typed transition system over a shared state record, treats execution outcomes as the sole acceptance signal, and reduces by configuration to a one-shot baseline and a single-agent execution-retry baseline—enabling controlled isolation of the contribution of multi-agent decomposition itself. Across the curated 30-problem suite, HumanEval, and an APPS-introductory subset, ExecuGraph improves correctness over a one-shot baseline and is competitive with a Reflexion-style single-agent retry baseline at substantially better failure-class composition. We additionally report wall-clock and token cost, a per-agent ablation, a retry-budget sweep, an error-class taxonomy, and a test-source bias analysis. Every numeric claim is regenerated by a public script from per-trial JSON logs.

Future work falls into five directions. First, we plan to integrate lightweight formal-verification probes (e.g. property-based tests via Hypothesis, type-state checks) as additional decision-predicate inputs alongside execution. Second, we plan to extend the sandbox and signature-aliasing layers to support non-Python languages, beginning with TypeScript and Java. Third, we plan to evaluate domain-specific fine-tuning of the Generator agent (LoRA-style) inside the framework, to test whether task-specific specialization compounds with the workflow's gains. Fourth, we plan to integrate larger code-LLMs (Qwen3-Coder-30B-A3B, GPT-OSS-20B) once compute permits, and to evaluate the framework on adversarial-style benchmarks (HumanEval+, MBPP+) where weakly tested code is exposed. Fifth, motivated by the JudgeSense findings on judge instability under paraphrase [17], we plan to construct a code-review-specific paraphrase set and report the Reviewer's Judge Sensitivity Score; this would convert the present qualitative argument for the advisory-only design into a quantitative one.

ACKNOWLEDGEMENT

The authors thank the maintainers of LangGraph, LangChain, ChromaDB, and Ollama for the open-source infrastructure that made this work practical. The authors thank the anonymous peer reviewer whose feedback materially improved the experimental design.

VIII. REPRODUCIBILITY

The code repository accompanying this paper is structured so that every result table can be regenerated from a single command. We summarize the key reproducibility surface here; full step-by-step instructions are in `REPRODUCIBILITY.md` in the repository.

a) Software environment.: Python 3.11, dependencies pinned in `pyproject.toml`. A `Dockerfile` is provided that builds a container with all dependencies including a CUDA-enabled PyTorch wheel and a pre-pulled Ollama model set. The container is independent of the user's local Ollama installation.

b) Models.: Local Ollama, with the following digests recorded in `REPRODUCIBILITY.md`: `qwen2.5:7b-instruct-q4_K_M`, `qwen2.5-coder:7b-instruct-q4_K_M`, and (for cross-model) `deepseek-coder-v2:16b-lite-instruct-q4_K_M`.

c) Seeds.: Seeds $\{0, \ldots, N-1\}$ are passed through to the LLM backend (where supported by the model server), to the LLM-test sampler, and to the trial-runner's iteration order.

d) Determinism caveats.: GPU floating-point non-determinism in the Ollama runtime means bit-exact reproducibility is not guaranteed across hardware; observed run-to-run variation in pass-rate at fixed seed is bounded by the reported standard deviations and does not change the qualitative conclusions.

e) Commands.: The full result set is regenerated by:

```
# 1. environment + models
docker build -t execugraph .
docker run --gpus all -v $(pwd)/results:/app/results
     execugraph \
   bash scripts/run_full_grid.sh
# 2. table generation
python -m execugraph.analysis.build_tables \
   results/<run_id> --out paper/tables
# 3. compile
cd paper && latexmk -pdf main.tex
```

f) Per-trial artifacts.: Every trial appends one JSON line to `results/<run_id>/trials.jsonl` with the fields enumerated in Section IV-F. A small example artifact is committed under `results/example_run/` so that the table-generation pipeline can be exercised without running the LLM.

REFERENCES

[1] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020.

[2] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[3] D. Hendrycks, S. Basart, S. Kadavath, M. Mazeika, A. Arora, E. Guo, C. Burns, S. Puranik, H. He, D. Song, and J. Steinhardt, "Measuring coding challenge competence with APPS," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2021.

[4] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of EMNLP*, 2020.

[5] Y. Wang, W. Wang, S. Joty, and S. C. H. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," *arXiv preprint arXiv:2109.00859*, 2021.

[6] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2022.

[7] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain-of-thought reasoning in language models," *arXiv preprint arXiv:2203.11171*, 2022.

[8] X. Chen, C. Liu, and D. Song, "Execution-guided neural program synthesis," OpenReview, 2018.

[9] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig, "PAL: Program-aided language models," *arXiv preprint arXiv:2211.10435*, 2022.

[10] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," *arXiv preprint arXiv:2303.11366*, 2023.

[11] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. D. Lago, T. Hubert, P. Choy, C. de Masson d'Autume, I. Babuschkin, X. Chen, P.-S. Huang, J. Welbl, S. Gowal, A. Cherepanov, J. Molloy, D. J. Mankowitz, E. S. Robson, P. Kohli, N. de Freitas, K. Kavukcuoglu, and O. Vinyals, "Competition-level code generation with AlphaCode," *Science*, 2022.

[12] A. Chowdhery *et al.*, "PaLM: Scaling language modeling with pathways," *arXiv preprint arXiv:2204.02311*, 2022.

[13] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "CodeGen: An open large language model for code with multi-turn program synthesis," *arXiv preprint arXiv:2203.13474*, 2022.

[14] D. Fried, A. Aghajanyan, J. Lin, S. Wang, E. Wallace, F. Shi, R. Zhong, W. tau Yih, L. Zettlemoyer, and M. Lewis, "InCoder: A generative model for code infilling and synthesis," *arXiv preprint arXiv:2204.05999*, 2022.

[15] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[16] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," *arXiv preprint arXiv:2304.05128*, 2023.

[17] JudgeSense Authors, "JudgeSense: A benchmark for prompt sensitivity in LLM-as-a-Judge systems," *arXiv preprint arXiv:2604.23478*, 2026, author list pending de-anonymization at time of writing; update on camera-ready. [Online]. Available: https://arxiv.org/abs/2604.23478

[18] D. Huang, J. M. Zhang, M. Luck, Q. Bu, Y. Qing, and H. Cui, "AgentCoder: Multi-agent-based code generation with iterative testing and optimisation," *arXiv preprint arXiv:2312.13010*, 2023.

[19] M. A. Islam, M. E. Ali, and M. R. Parvez, "MapCoder: Multi-agent code generation for competitive problem solving," *arXiv preprint arXiv:2405.11403*, 2024.

[20] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta programming for a multi-agent collaborative framework," *arXiv preprint arXiv:2308.00352*, 2023.

[21] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen llm applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.

[22] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2023.

[23] T. B. Richards *et al.*, "Auto-GPT: An autonomous GPT-4 experiment," https://github.com/Significant-Gravitas/Auto-GPT, 2023.

[24] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, "AgentBench: Evaluating LLMs as agents," *arXiv preprint arXiv:2308.03688*, 2023.

[25] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," *arXiv preprint arXiv:2302.04761*, 2023.

[26] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020.

[27] Chroma Authors, "Chroma: The AI-native open-source embedding database," https://docs.trychroma.com/, 2024, open-source software.

[28] LangChain Authors, "LangChain: Building applications with LLMs through composability," https://docs.langchain.com/, 2024, open-source software.

[29] LangGraph Authors, "LangGraph: A library for building stateful, multi-actor applications with LLMs," https://github.com/langchain-ai/langgraph, 2024, open-source software.

[30] Ollama Authors, "Ollama: Run large language models locally," https://ollama.com/, 2024, open-source software.

[31] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu, K. Dang, Y. Fan, Y. Zhang, A. Yang, R. Men, F. Huang, B. Zheng, Y. Miao, S. Quan, Y. Feng, X. Ren, X. Ren, J. Zhou, and J. Lin, "Qwen2.5-Coder technical report," *arXiv preprint arXiv:2409.12186*, 2024.

[32] DeepSeek-AI, "DeepSeek-Coder-V2: Breaking the barrier of closed-source models in code intelligence," *arXiv preprint arXiv:2406.11931*, 2024.

[33] OpenAI, "gpt-oss: An open-weight language model from OpenAI," Apache-2.0 weights at https://huggingface.co/openai/gpt-oss-20b, 2025.

[34] Qwen Team, "Qwen3-Coder: Agentic code generation with mixture-of-experts," Technical report; weights at https://huggingface.co/Qwen/Qwen3-Coder-30B-A3B-Instruct, 2025.